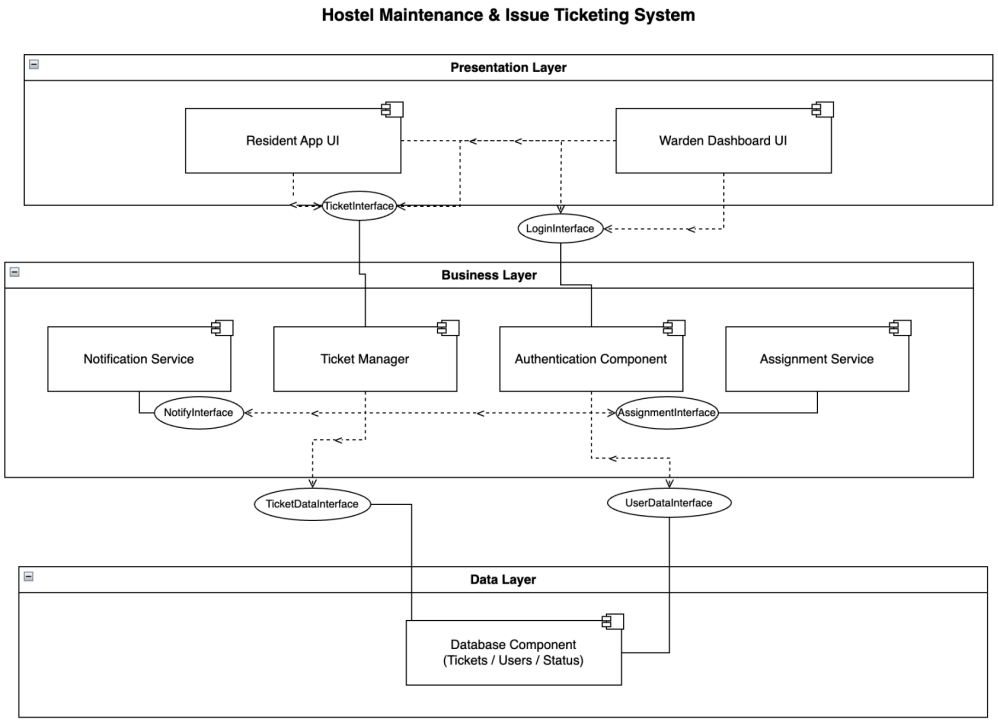
Lab 3: Component Modelling & Architectural Pattern Selection

NAME:- HEMAVATHI E
SRN:- PES2UG24CS193
Sem -V
SECTION :- D

1.Component Diagram



2.Written Justification

Architecture Selection

We chose Layered Architecture for the Hostel Maintenance & Issue Ticketing System.

Reason 1: Clear Separation of Concerns
The system serves two distinct actor groups — Hostel Residents, who raise and track maintenance tickets through the Resident App UI, and Maintenance Wardens, who review, assign, and resolve them through the Warden Dashboard UI. A layered structure cleanly separates the Presentation Layer (Resident App UI, Warden Dashboard UI), the Business Layer (Ticket Manager, Authentication Component, Notification Service, Assignment Service), and the Data Layer (Database Component). This keeps the UI, the ticketing rules, and the stored data independent, so each can be modified (e.g., redesigning the resident app, or swapping the notification channel from SMS to email) without disturbing the others.

Reason 2: Appropriate Scale for a Campus System
This is a single-institution, moderate-traffic system used within one hostel campus, not a globally distributed service. A Layered Architecture provides sufficient modularity and maintainability for this scale without the operational overhead of managing independently deployed microservices (service discovery, inter-service networking, distributed data consistency), which would be excessive for a hostel-scale ticketing tool with a small, well-defined set of features.

Security Advantage
By isolating the Authentication Component within the Business Layer, both the Resident App UI and the Warden Dashboard UI must pass through the LoginInterface before reaching the Ticket Manager, and the Presentation Layer never accesses the Data Layer directly — only the Authentication Component and Ticket Manager are permitted to call the Database Component, via the UserDataInterface and TicketDataInterface respectively. This reduces the attack surface, since sensitive hostel resident data (room numbers, complaint history) can only be reached through validated, controlled interfaces rather than being directly exposed to the UI layer.

Performance Benefit
Because each layer only communicates with the layer directly below it through well-defined interfaces, the Business Layer can cache frequently accessed data (such as room lists and issue categories) and batch database calls from the Ticket Manager, reducing redundant queries to the Database Component. Offloading notifications and assignment logic into their own components (Notification Service, Assignment Service) also means the Ticket Manager is not blocked waiting on a notification or an assignment decision — it can hand the work off through the NotifyInterface and AssignmentInterface and continue processing the next request. This layered separation also makes it straightforward to scale the Business Layer independently if ticket volume grows during peak periods such as the start of a semester, without having to change the Presentation or Data layers.